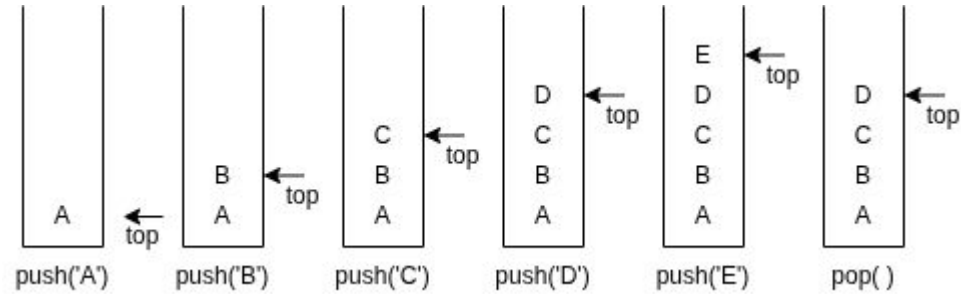


Lecture 14

Stack

- A stack is an Linear data structure in which insertions (also called pushes & adds) and deletions (also called pops & removes) are made at one end called the top.
- Given a stack $S = (a_0, a_1, \dots, a_{n-1})$, we say a_0 is the bottom element and a_{n-1} is the top element, a_i is on top of element a_{i-1} , $0 < i < n$.
- The restrictions on the stack imply that if we add the elements A, B, C, D, E to the stack in that order then E is the first element we delete from the stack.
- Since the last element inserted into is the first element removed, a stack is also known as Last-In-First-Out (LIFO) list.

Inserting and deleting elements in a stack



example

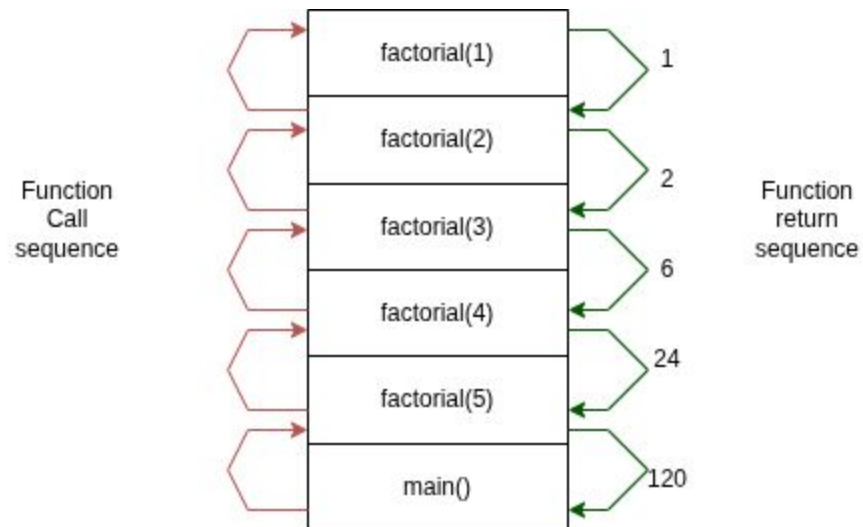
Recursive functions are implemented using a stack.

- Finding the factorial of a given number.

- ```
int factorial(int num){
 if(num == 0 || num == 1)
 return 1;
 return num * factorial(num - 1);
}
```

- Calling the factorial function , factorial(5) will return 120.

continue...



# Stack Creation

```
#define MAX_STACK_SIZE 5

typedef struct{
 int key;
}element;

element stack[MAX_STACK_SIZE];

int top = -1;
```

# Stack Operations

1. Push
2. Pop
3. isFull
4. isEmpty

# Push

```
push(element):
```

```
 /* check for stack overflow */
```

```
 if(top == size - 1):
```

```
 stack is full
```

```
 return
```

```
 top++;
```

```
 stack[top] = element
```

# Pop

pop():

```
/* check for stack underflow */
```

```
if(top == -1):
```

```
 print stack is underflow
```

```
char ch;
```

```
ch = stack[top]
```

```
top--
```

```
return removed element
```

# isFull

```
isFull():
```

```
 if(top == size - 1):
```

```
 return true
```

```
 return false
```

# isEmpty

```
isEmpty():
```

```
 if(top == -1):
```

```
 return true;
```

```
 return false;
```

# Problem

Given the elements present in the stack pop the middle element. The size of stack is 5. The first stack in the given picture is input and the second stack is output.

